Agentic QA Command Center — Revised PRD

Free-first, single-agent, evidence-backed agentic testing system

Version 2.0 | Proposed implementation baseline | Free-first | Primary targets: simple HTML app + isolated OWASP Juice Shop

1. Executive Summary

Build a single-agent, CLI-first QA system that receives a natural-language objective, selects permission-gated browser tools, captures evidence on every action, verifies outcomes with deterministic assertions, and produces a JSONL trace plus static HTML report. The LLM proposes what to investigate; deterministic systems establish what happened.

2. Core Scope

- MUST: single QA agent, provider abstraction, Playwright tool gateway, deterministic assertions, evidence capture, JSONL trace, RunState, budgets, loop detection, recovery, blind evaluation, static report, prompt-injection fixture, simple HTML target.
- NEED: isolated Juice Shop target, regression store, GitHub Actions, HTTP/API tools, plugin interfaces.
- Removed/deferred: Wish tier, multi-agent architecture, live dashboard, RBAC, hosted SaaS, Braintrust dependency, self-hosted NIM requirement, broad arbitrary-target security automation.

3. Free-First Provider Plan

Use Gemini Developer API as the default development provider and NVIDIA NIM as a second provider behind the same interface. Google currently documents a free tier for selected Gemini models, while NVIDIA documents free NIM API access for Developer Program prototyping and free downloadable NIM access for research/development/testing.
■cite■turn0search0■turn0search1■turn1search0■turn1search2■

Do not make either vendor the architecture. Enforce model-call and run budgets so the project remains viable on free quotas.

4. Architecture

Natural-language objective → QAAgent → Tool Gateway → Playwright/API tools → Evidence Layer → Deterministic Assertion Engine → JSONL Trace → Static HTML Report. RunState is passed through every agent turn. The gateway is the only route to tools.

5. Reliability

- Action fingerprint = hash(tool + normalized args + URL); observation fingerprint = hash(relevant DOM/state).
- Three repeated action/observation pairs trigger recovery.
- Recovery: re-observe → validate state → bounded retry/alternate locator → continue or terminate INCONCLUSIVE.
- Default budgets: 25 actions, 40 model calls, 3 minutes per run.
- Transient Playwright failures use bounded retries; a retry that passes is recorded as PASS_AFTER_RETRY/FLAKY rather than silently becoming PASS.

6. Evaluation

Use a fixed blind benchmark based on the known defects in the existing HTML app. The agent is not told the defect descriptions. Score discovery rate, evidence support rate, false positives, action efficiency, loop rate, recovery success, and run validity. Initial target: discover at least 4/5 known defects with zero unsupported defect claims across the benchmark definition.

7. Safety

- Application content is untrusted data, not instructions.
- Observed DOM/text is delimited and cannot modify system policy, target, permissions or budgets.
- Target allowlist is mandatory.
- Secrets are scrubbed from traces.
- Destructive tools require explicit permission and human approval where applicable.

- Juice Shop is isolated and authorized; ZAP is added only after the core is stable.

8. Roadmap

Phase 0 — Foundation: Provider interface, Gemini adapter, NIM adapter, tool interface, gateway, JSONL trace, RunState, basic tests. *Exit:* Working fake-tool trace and provider contract tests.

Phase 1 — MVP: Playwright tools, deterministic assertions, evidence capture, CLI, static report, simple HTML target. *Exit:* Natural-language objective produces evidence-backed PASS/FAIL/INCONCLUSIVE report.

Phase 2 — Agent Reliability + Blind Eval: Loop detection, budgets, recovery, five-defect hidden benchmark, scoring and hallucination checks. *Exit:* Measured discovery rate, false positives, efficiency and recovery.

Phase 3 — Regression + CI: Regression store, stable test IDs, GitHub Actions. *Exit:* Confirmed defect can fail CI if reintroduced.

Phase 4 — Safety Hardening: Prompt-injection fixture, target allowlist, SSRF controls for any network tool, secret scrubbing, destructive-action approval. *Exit:* Adversarial suite passes without objective hijacking.

Phase 5 — Juice Shop: Isolated Juice Shop target, security profile, later ZAP integration. *Exit:* Same agent architecture works against second target and produces evidence-backed security findings.

Phase 6 — Optional extensions: HTTP/API tools, plugin documentation, second toy app, richer report. *Exit:* Only implement when the core demonstrates a concrete need.

9. Portfolio / Hiring Evidence

- Live hidden-defect discovery demo.
- Benchmark metrics and repeatability report.
- Evidence-backed trace showing why a defect was reported.
- Prompt-injection adversarial test.
- Regression test generated from a confirmed discovery and enforced in CI.
- Same core runtime demonstrated against simple HTML app and isolated Juice Shop.

10. Final Decision

Do not build a platform. Build the controlled agentic testing loop first and prove it quantitatively. The reviewed PRD's strongest ideas—deterministic verification, evidence-first design, blind evaluation, tool-gateway enforcement and regression—remain central. ■filecite■turn0file0■L194-L211■